Unidad 10 · SQL con varias tablas: joins y agregación

Qué vas a poder hacer. Juntar tablas siguiendo sus claves foráneas sin fabricar filas de más, conservar con `LEFT JOIN` lo que no tiene pareja y encontrar lo que falta, resumir con `COUNT`, `SUM`, `AVG`, `MIN` y `MAX` por grupos, filtrar antes de agrupar (`WHERE`) o después (`HAVING`), y leer una consulta larga cláusula por cláusula en el orden en que el motor la evalúa.

Lo que necesitas antes. La unidad 9 completa: `SELECT`, `WHERE`, `NULL`, `ORDER BY` y `LIMIT`. De la unidad 5, qué es una clave primaria y una foránea. De la unidad 7, el producto cartesiano (×) y la junta (⋈) del álgebra relacional.

Tiempo. Unas 6 horas: 2,5 de lectura con SQLite abierto y 3,5 de práctica. Todo se corre con `sql/u10_joins.sql` (la liga del ejemplo) y `sql/u10_caso.sql` (sobre `datos/estampa_sur.db`).

> **Cómo leer los resultados.** Como en la unidad 9, cada consulta se ejecutó en SQLite 3.50 y su resultado aparece tal como lo entrega el motor: sin punto de miles y con punto decimal. En la lectura, las cifras van en formato chileno.

1. La idea: la respuesta está repartida

La normalización (unidad 6) reparte los datos para no guardar nada dos veces: el nombre del equipo vive en `equipo`, el jugador apunta a su equipo con una clave foránea, y el gol apunta al jugador. El precio de eso es que casi ninguna pregunta interesante cabe en una tabla. «¿Cuántos goles hizo cada equipo?» necesita los goles, saber de quién es cada gol y saber de qué equipo es cada jugador.

Un **join** reconstruye, en el momento de consultar, las filas que la normalización separó. Y la **agregación** las resume: de 13 goles sueltos a una fila por equipo.

La liga de la unidad. Una liga vecinal de baby fútbol de Chillán, con partidos de dos tiempos de 30 minutos. Las Termas se acaba de inscribir y todavía no tiene jugadores; Benjamín Lara se inscribió sin equipo (`id_equipo` en `NULL`).

id_equipo	nombre	comuna
1	Los Robles	Chillán
2	Villa Alegre	Chillán Viejo
3	El Bajo	Chillán
4	Las Termas	Pinto

id_jugador	nombre	puesto	id_equipo
1	Bastián Rojas	arquero	1
2	Felipe Sáez	delantero	1
3	Ignacio Lagos	defensa	1
4	Tomás Vidal	delantero	2
5	Cristóbal Mora	arquero	2
6	Diego Parra	defensa	2
7	Joaquín Reyes	delantero	3
8	Martín Cofré	defensa	3
9	Nicolás Bravo	arquero	3
10	Benjamín Lara	delantero	NULL

id_gol	id_jugador	jornada	minuto
1	2	1	12
2	2	1	38
3	4	1	20
4	7	1	44
5	4	2	5
6	6	2	31
7	2	2	47
8	7	2	50
9	3	3	18
10	4	3	26
11	7	3	39
12	4	3	41
13	8	3	49

2. El producto cartesiano: lo que pasa si no se dice cómo juntar

Si se nombran dos tablas en el `FROM` sin decir cómo se relacionan, el motor combina **cada fila de una con cada fila de la otra**. Es el producto cartesiano de la unidad 7:

$$|A \times B| = |A| \cdot |B|$$

```
SELECT COUNT(*) AS filas FROM equipo, jugador;
```

filas
40

Cuatro equipos por diez jugadores, 40 filas. La mayoría no significa nada: dice que Bastián Rojas juega en los cuatro equipos.

```
SELECT j.nombre AS jugador, e.nombre AS equipo
FROM jugador j, equipo e
WHERE j.id_jugador <= 2
ORDER BY j.id_jugador, e.id_equipo;
```

jugador	equipo
Bastián Rojas	Los Robles
Bastián Rojas	Villa Alegre
Bastián Rojas	El Bajo
Bastián Rojas	Las Termas
Felipe Sáez	Los Robles
Felipe Sáez	Villa Alegre
Felipe Sáez	El Bajo
Felipe Sáez	Las Termas

Lo mismo pasa con un `JOIN` al que le falta su condición. SQLite y MySQL lo aceptan sin reclamar (el estándar exige el `ON`):

```
SELECT COUNT(*) AS filas FROM equipo JOIN jugador;
```

filas
40

Por qué importa. En la base del caso, `cliente` tiene 30 filas y `pedido` 120. Su producto cartesiano tiene 3.600 filas (sección 14). Una suma de ventas sobre ese producto sale 30 veces más grande que la real, sin error y sin aviso. La defensa es la regla de la sección 5: **con n tablas hacen falta $n - 1$ condiciones de junta**, y contar las filas antes de sumar.

---

3. `INNER JOIN`: juntar por la clave foránea

Un join es un producto cartesiano del que se conservan solo las combinaciones que cumplen la condición. La condición casi siempre iguala una clave foránea con la clave primaria a la que apunta:

```
SELECT j.nombre AS jugador, e.nombre AS equipo
FROM jugador j
INNER JOIN equipo e ON e.id_equipo = j.id_equipo
ORDER BY e.nombre, j.nombre;
```

jugador	equipo
Joaquín Reyes	El Bajo
Martín Cofré	El Bajo
Nicolás Bravo	El Bajo
Bastián Rojas	Los Robles
Felipe Sáez	Los Robles
Ignacio Lagos	Los Robles
Cristóbal Mora	Villa Alegre
Diego Parra	Villa Alegre
Tomás Vidal	Villa Alegre

Nueve filas: cada jugador con su equipo. Faltan dos cosas, y es lo que define un `INNER JOIN`: Las Termas no aparece (ningún jugador apunta a ella) y Benjamín Lara tampoco (su `id_equipo` es `NULL`, y `NULL = 1` no es verdadero). **Un `INNER JOIN` deja solo las filas que tienen pareja en los dos lados.** `JOIN` a secas es lo mismo que `INNER JOIN`.

Alias de tabla. `jugador j` le pone a la tabla el nombre corto `j`, y `j.nombre` dice de qué tabla es la columna. Hace falta cuando las dos tablas tienen una columna con el mismo nombre (`nombre` existe en las dos); sin el prefijo, SQLite responde `ambiguous column name: nombre`. El alias va **después** del nombre de la tabla y **delante** de la columna: `jugador.j` o `nombre.j` no son lo mismo (el primero es la tabla `j` de una base llamada `jugador`).

El estilo antiguo: coma y `WHERE`. Antes de 1992 el join se escribía con las tablas separadas por coma y la condición en el `WHERE`. Da exactamente lo mismo:

```
SELECT j.nombre AS jugador, e.nombre AS equipo
FROM jugador j, equipo e
WHERE e.id_equipo = j.id_equipo
ORDER BY e.nombre, j.nombre;
```

jugador	equipo
Joaquín Reyes	El Bajo
Martín Cofré	El Bajo
Nicolás Bravo	El Bajo
Bastián Rojas	Los Robles
Felipe Sáez	Los Robles
Ignacio Lagos	Los Robles
Cristóbal Mora	Villa Alegre
Diego Parra	Villa Alegre
Tomás Vidal	Villa Alegre

Se sigue viendo mucho, y conviene saber leerlo. Pero el `JOIN ... ON` es mejor por dos razones: la condición queda pegada a la tabla que junta (con cinco tablas, se ve cuál falta), y el `WHERE` queda solo para filtrar. Olvidar una condición en el estilo de coma es el producto cartesiano de la sección 2.

`USING` y `NATURAL JOIN`. `JOIN equipo USING (id_equipo)` sirve cuando la columna se llama igual en las dos tablas. `NATURAL JOIN` junta por **todas** las columnas de igual nombre, y aquí juntaría también por `nombre` (el del jugador con el del equipo): no lo uses.

---

4. `LEFT JOIN`: conservar lo que no tiene pareja

`A LEFT JOIN B ON ...` hace el `INNER JOIN` y además **agrega cada fila de A que no encontró pareja**, con `NULL` en todas las columnas de B:

```
SELECT e.nombre AS equipo, j.nombre AS jugador
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
ORDER BY e.nombre, j.nombre;
```

equipo	jugador
El Bajo	Joaquín Reyes
El Bajo	Martín Cofré
El Bajo	Nicolás Bravo
Las Termas	NULL
Los Robles	Bastián Rojas
Los Robles	Felipe Sáez
Los Robles	Ignacio Lagos
Villa Alegre	Cristóbal Mora
Villa Alegre	Diego Parra
Villa Alegre	Tomás Vidal

Las Termas aparece una vez, con jugador `NULL`. «Izquierda» es la tabla escrita antes del `JOIN`: la que se conserva entera.

Encontrar lo que falta (anti-join). Las filas agregadas son justo las que tienen `NULL` en la clave de B. Filtrarlas responde «¿qué equipos no tienen jugadores?»:

```
SELECT e.nombre AS equipo
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
WHERE j.id_jugador IS NULL;
```

equipo
Las Termas

Se pregunta por la clave primaria de B (`j.id_jugador`), que nunca es `NULL` en una fila real: si es `NULL`, la fila la puso el `LEFT JOIN`. Es el patrón de «clientes sin pedidos» del caso.

`RIGHT JOIN` conserva la tabla de la derecha. Es un `LEFT JOIN` con las tablas al revés, y casi siempre se lee mejor escrito como `LEFT`. Los delanteros con su equipo, si lo tienen:

```
SELECT e.nombre AS equipo, j.nombre AS jugador
FROM equipo e
RIGHT JOIN jugador j ON j.id_equipo = e.id_equipo
WHERE j.puesto = 'delantero'
ORDER BY j.nombre;
```

equipo	jugador
NULL	Benjamín Lara
Los Robles	Felipe Sáez
El Bajo	Joaquín Reyes
Villa Alegre	Tomás Vidal

`FULL OUTER JOIN` conserva las dos. Con el filtro de abajo deja solo lo que no tiene pareja en ningún lado: el equipo sin jugadores y el jugador sin equipo.

```
SELECT e.nombre AS equipo, j.nombre AS jugador
FROM equipo e
FULL OUTER JOIN jugador j ON j.id_equipo = e.id_equipo
WHERE e.id_equipo IS NULL OR j.id_jugador IS NULL
ORDER BY equipo;
```

equipo	jugador
NULL	Benjamín Lara
Las Termas	NULL

SQLite tiene `RIGHT` y `FULL` desde la versión 3.39 (2022); con una versión anterior responde que no los soporta. **MySQL no tiene `FULL OUTER JOIN`.** Se arma con dos `LEFT JOIN` unidos por `UNION` (que junta los resultados de dos consultas y elimina repetidos); corre igual en SQLite:

```
SELECT e.nombre AS equipo, j.nombre AS jugador
FROM equipo e LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
WHERE j.id_jugador IS NULL
UNION
SELECT e.nombre AS equipo, j.nombre AS jugador
FROM jugador j LEFT JOIN equipo e ON e.id_equipo = j.id_equipo
WHERE e.id_equipo IS NULL
ORDER BY equipo;
```

equipo	jugador
NULL	Benjamín Lara
Las Termas	NULL

La trampa del `LEFT JOIN`: filtrar la tabla de la derecha en el `WHERE`. Se quiere cada equipo con su arquero, o `NULL` si no tiene. Escrito con el filtro en el `WHERE`:

```
SELECT e.nombre AS equipo, j.nombre AS arquero
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
WHERE j.puesto = 'arquero'
ORDER BY e.nombre;
```

equipo	arquero
El Bajo	Nicolás Bravo
Los Robles	Bastián Rojas
Villa Alegre	Cristóbal Mora

Las Termas desapareció. El `LEFT JOIN` la agregó con `j.puesto` en `NULL`, y después el `WHERE` la eliminó, porque `NULL = 'arquero'` no es verdadero: el `LEFT` quedó convertido en `INNER`. Una condición sobre la tabla de la derecha va en el `ON`, que decide **qué cuenta como pareja**:

```
SELECT e.nombre AS equipo, j.nombre AS arquero
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo AND j.puesto = 'arquero'
ORDER BY e.nombre;
```

equipo	arquero
El Bajo	Nicolás Bravo
Las Termas	NULL
Los Robles	Bastián Rojas
Villa Alegre	Cristóbal Mora

Regla: en un `LEFT JOIN`, **lo que filtra la tabla de la izquierda va en el `WHERE`; lo que filtra la de la derecha, en el `ON`**. En un `INNER JOIN` da lo mismo.

---

5. Juntar más de dos tablas

Cada `JOIN` agrega una tabla al resultado que se lleva hasta ese momento. Se sigue el camino de las claves foráneas, como en el diagrama de tablas de las unidades 4 y 5:

```
gol.id_jugador  ->  jugador.id_jugador
jugador.id_equipo  ->  equipo.id_equipo
```

Los goles de la jornada 3, con el nombre del jugador y de su equipo:

```
SELECT g.minuto, j.nombre AS jugador, e.nombre AS equipo
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_equipo
WHERE g.jornada = 3
ORDER BY g.minuto;
```

minuto	jugador	equipo
18	Ignacio Lagos	Los Robles
26	Tomás Vidal	Villa Alegre
39	Joaquín Reyes	El Bajo
41	Tomás Vidal	Villa Alegre
49	Martín Cofré	El Bajo

Cuántas filas esperar. Si se junta desde el lado «muchos» hacia el lado «uno» (gol → jugador → equipo), cada fila encuentra a lo más una pareja y el resultado tiene **tantas filas como la primera tabla** (13 goles, 13 filas antes del `WHERE`). Si se junta desde el «uno» hacia el «muchos» (equipo → jugador), las filas se multiplican. Eso no es un error, pero hay que saberlo antes de sumar (ejercicio 10.9).

---

6. Funciones de agregación

Una función de agregación toma **muchas filas y devuelve un valor**:

Función	Qué devuelve	Con `NULL`
`COUNT(*)`	cuántas filas hay	cuenta todas las filas
`COUNT(col)`	cuántas filas tienen `col` no nula	ignora los `NULL`
`COUNT(DISTINCT col)`	cuántos valores distintos hay	ignora los `NULL`
`SUM(col)`	la suma	ignora los `NULL`; si todos son `NULL`, da `NULL`
`AVG(col)`	el promedio de los valores no nulos	los `NULL` no cuentan ni arriba ni abajo
`MIN(col)`, `MAX(col)`	el menor y el mayor	ignora los `NULL`

```
SELECT COUNT(*) AS goles, COUNT(DISTINCT id_jugador) AS goleadores,
       MIN(minuto) AS primero, MAX(minuto) AS ultimo, ROUND(AVG(minuto), 1) AS minuto_medio
FROM gol;
```

goles	goleadores	primero	ultimo	minuto_medio
13	6	5	50	32.3

La diferencia entre `COUNT(*)`, `COUNT(col)` y `COUNT(DISTINCT col)`, en una sola consulta:

```
SELECT COUNT(*) AS jugadores, COUNT(id_equipo) AS con_equipo,
       COUNT(DISTINCT id_equipo) AS equipos_con_jugadores
FROM jugador;
```

jugadores	con_equipo	equipos_con_jugadores
10	9	3

Diez jugadores, nueve con equipo (Lara no cuenta) y tres equipos distintos (Las Termas no tiene a nadie). El promedio de una columna con `NULL` no es la suma dividida por todas las filas: es la suma dividida por las filas **no nulas**. En SQLite, `AVG` siempre devuelve un número real; MySQL devuelve un decimal con cuatro cifras.

---

7. `GROUP BY`: una fila por grupo

`GROUP BY` parte las filas en grupos con el mismo valor en las columnas indicadas, y las funciones de agregación se calculan **dentro de cada grupo**. Los goles por equipo:

```
SELECT e.nombre AS equipo, COUNT(*) AS goles
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_equipo
GROUP BY e.nombre
ORDER BY goles DESC, equipo;
```

equipo	goles
Villa Alegre	5
El Bajo	4
Los Robles	4

Contar ceros: `LEFT JOIN` y `COUNT(col)`. Un `INNER JOIN` solo produce grupos para lo que tiene filas: los jugadores sin goles no aparecen. Para que aparezcan con 0, la tabla que se quiere completa va a la izquierda de un `LEFT JOIN`, y se cuenta una columna de la derecha (que vale `NULL` en las filas agregadas), no `*`:

```
SELECT j.nombre AS jugador, COUNT(g.id_gol) AS goles
FROM jugador j
LEFT JOIN gol g ON g.id_jugador = j.id_jugador
GROUP BY j.id_jugador, j.nombre
ORDER BY goles DESC, jugador;
```

jugador	goles
Tomás Vidal	4
Felipe Sáez	3
Joaquín Reyes	3
Diego Parra	1
Ignacio Lagos	1
Martín Cofré	1
Bastián Rojas	0
Benjamín Lara	0
Cristóbal Mora	0
Nicolás Bravo	0

Con `COUNT(*)` los cuatro de abajo saldrían con 1: la fila que agregó el `LEFT JOIN` existe, aunque esté llena de `NULL` (ejercicio 10.8).

Se agrupa por la clave. El `GROUP BY j.id_jugador, j.nombre` agrupa por la clave primaria y agrega el nombre para poder mostrarlo. Si se agrupara solo por `j.nombre`, dos jugadores homónimos quedarían sumados en uno.

La regla del `SELECT` con `GROUP BY`. Después de agrupar queda **una fila por grupo**. Por eso cada columna del `SELECT` tiene que ser una de las del `GROUP BY` o estar dentro de una función de agregación. Una columna suelta (por ejemplo `j.nombre` en una consulta agrupada por equipo) no tiene un valor único por grupo: ¿cuál de los tres jugadores del equipo se muestra?

- **MySQL** (desde la versión 5.7, con el modo `ONLY_FULL_GROUP_BY`, activo por omisión) rechaza la consulta con el error 1055: `Expression #2 of SELECT list is not in GROUP BY clause and contains nonaggregated column ...`.
- **SQLite** la acepta y muestra el valor de **alguna** fila del grupo, sin aviso. El resultado parece correcto y no lo es (ejercicio 10.8).

---

8. `WHERE` contra `HAVING`

Los dos filtran, pero en momentos distintos:

	`WHERE`	`HAVING`
Cuándo actúa	antes de agrupar	después de agrupar
Qué filtra	filas	grupos
Puede usar `COUNT`, `SUM`...	no	sí
Ejemplo	`WHERE g.minuto > 30` (solo los goles del segundo tiempo)	`HAVING COUNT(*) >= 3` (solo los grupos con 3 o más)

Una agregación en el `WHERE` es un error, porque cuando el `WHERE` actúa los grupos todavía no existen:

```
SELECT id_jugador, COUNT(*) AS goles
FROM gol
WHERE COUNT(*) >= 3
GROUP BY id_jugador;
```

Error: misuse of aggregate: COUNT()

MySQL responde `Invalid use of group function` (error 1111). Con `HAVING`:

```
SELECT id_jugador, COUNT(*) AS goles
FROM gol
GROUP BY id_jugador
HAVING COUNT(*) >= 3
ORDER BY goles DESC, id_jugador;
```

id_jugador	goles
4	4
2	3
7	3

Y los dos juntos: los equipos que marcaron 3 o más goles **en el segundo tiempo**. El `WHERE` deja solo los goles después del minuto 30; el `GROUP BY` los cuenta por equipo; el `HAVING` deja los grupos que llegan a 3.

```
SELECT e.nombre AS equipo, COUNT(*) AS goles_2t
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_equipo
WHERE g.minuto > 30
GROUP BY e.nombre
HAVING COUNT(*) >= 3;
```

equipo	goles_2t
El Bajo	4

Una condición que no usa agregaciones (`g.minuto > 30` , `e.nombre <> 'El Bajo'`) podría ir en cualquiera de los dos, pero va en el `WHERE` : filtra antes, el motor agrupa menos filas y se lee mejor.

---

9. El orden completo de evaluación

Con todo lo de esta unidad, una consulta se evalúa así:

Paso	Cláusula	Qué hace	Qué se puede usar
1	`FROM` y cada `JOIN ... ON`	arma las filas combinadas	columnas de las tablas
2	`WHERE`	filtra filas	columnas; **no** agregaciones ni alias
3	`GROUP BY`	forma los grupos	columnas
4	`HAVING`	filtra grupos	agregaciones y columnas del `GROUP BY`
5	`SELECT`	calcula una fila por grupo y pone los alias	columnas del `GROUP BY` y agregaciones
6	`DISTINCT`	elimina repetidos	
7	`ORDER BY`	ordena	alias, agregaciones
8	`LIMIT`	corta	

Ante una consulta que da un número raro, recórrela en este orden y pregúntate en cada paso cuántas filas hay. Casi siempre el problema está en el paso 1 (un join que multiplica) o en el 2 (un `WHERE` que anuló un `LEFT JOIN`).

---

10. Ejemplo resuelto · El boletín de la liga

La directiva publica cada semana un boletín con dos tablas: los goles de la última jornada por equipo (los cuatro, aunque alguno no haya marcado) y los goleadores del torneo con 2 o más goles.

Tabla 1 · Goles de la jornada 3 por equipo. Que salgan los cuatro equipos exige partir de `equipo` con `LEFT JOIN`, y que la condición de la jornada vaya en el `ON` del gol:

```
SELECT e.nombre AS equipo, COUNT(g.id_gol) AS goles_j3
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
LEFT JOIN gol g     ON g.id_jugador = j.id_jugador AND g.jornada = 3
GROUP BY e.nombre
ORDER BY goles_j3 DESC, equipo;
```

equipo	goles_j3
El Bajo	2
Villa Alegre	2
Los Robles	1
Las Termas	0

Cláusula por cláusula:

1. `FROM equipo` y el primer `LEFT JOIN`: cada equipo con cada uno de sus jugadores; Las Termas, con un jugador `NULL`.
2. El segundo `LEFT JOIN`: a cada jugador, sus goles **de la jornada 3**; quien no marcó esa jornada queda con gol `NULL`, pero su fila se conserva.

3. `GROUP BY e.nombre`: un grupo por equipo, los cuatro.
4. `SELECT`: `COUNT(g.id_gol)` cuenta solo los goles reales; las filas con gol `NULL` suman 0.
5. `ORDER BY`: el que más marcó arriba; el nombre desempata.

Si la jornada se filtra en el `WHERE`, Las Termas se pierde, igual que el arquero de la sección 4:

```
SELECT e.nombre AS equipo, COUNT(g.id_gol) AS goles_j3
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
LEFT JOIN gol g     ON g.id_jugador = j.id_jugador
WHERE g.jornada = 3
GROUP BY e.nombre
ORDER BY goles_j3 DESC, equipo;
```

equipo	goles_j3
El Bajo	2
Villa Alegre	2
Los Robles	1

Tabla 2 · Goleadores con 2 o más goles. Además del total, la directiva quiere cuántos fueron en el segundo tiempo. Eso se cuenta con un `CASE` dentro de un `SUM`: vale 1 si el gol fue después del minuto 30 y 0 si no.

```
SELECT j.nombre AS jugador, e.nombre AS equipo, COUNT(*) AS goles,
       SUM(CASE WHEN g.minuto > 30 THEN 1 ELSE 0 END) AS en_2t
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_equipo
GROUP BY j.id_jugador, j.nombre, e.nombre
HAVING COUNT(*) >= 2
ORDER BY goles DESC, jugador;
```

jugador	equipo	goles	en_2t
Tomás Vidal	Villa Alegre	4	1
Felipe Sáez	Los Robles	3	2
Joaquín Reyes	El Bajo	3	3

Cuántas filas hay en cada paso:

Paso	Filas o grupos	Por qué
`FROM gol` y los dos `JOIN`	13 filas	se junta desde el lado «muchos»: una fila por gol
`GROUP BY`	6 grupos	uno por cada jugador que marcó
`HAVING COUNT(*) >= 2`	quedan 3	Parra, Lagos y Cofré tienen 1 gol
`ORDER BY`	las mismas 3, ordenadas	

`e.nombre` está en el `GROUP BY` aunque no cambie dentro del grupo (cada jugador tiene un equipo): así la consulta corre en MySQL con `ONLY_FULL_GROUP_BY`.

Cómo se lee.

- **La jornada 3 fue pareja**: El Bajo y Villa Alegre marcaron 2 cada uno. Las Termas aparece con 0 y no por error: es el equipo que todavía no inscribe jugadores, y la directiva tiene que cerrar eso antes de programarle partidos.
- **Joaquín Reyes hizo sus tres goles en el segundo tiempo**; Tomás Vidal, el goleador, solo uno. Si el boletín destaca a un jugador «decisivo», los números apuntan a Reyes.
- Cuatro jugadores inscritos no han marcado: Bastián Rojas, Cristóbal Mora, Nicolás Bravo y Benjamín Lara. Tres son arqueros; el cuarto no tiene equipo.

11. Errores típicos

Error	Cómo evitarlo
Producto cartesiano: `FROM a, b` o `JOIN` sin `ON`	con n tablas, $n-1$ condiciones; cuenta las filas antes de sumar
Condición de junta con la columna equivocada (`e.id_equipo = j.id_jugador`)	la foránea con **su** primaria; el resultado se ve creíble y está mal
Condición sobre la tabla derecha de un `LEFT JOIN` en el `WHERE`	va en el `ON`; en el `WHERE` convierte el `LEFT` en `INNER`
`COUNT(*)` después de un `LEFT JOIN` para contar «cuántos tiene»	`COUNT(columna de la derecha)`: los que no tienen salen con 0 y no con 1
Columna en el `SELECT` que no está en el `GROUP BY` ni agregada	MySQL da error 1055; SQLite muestra un valor cualquiera del grupo
`COUNT`, `SUM` o `AVG` en el `WHERE`	las condiciones sobre grupos van en `HAVING`
Contar filas de una junta como si fueran entidades (`COUNT(*)` de pedido con detalle cuenta líneas)	`COUNT(DISTINCT p.id_pedido)`
Agrupar por el nombre en vez de la clave	`GROUP BY` la clave primaria y el nombre
`NATURAL JOIN`	junta por todas las columnas de igual nombre; usa `ON`

12. Práctica

Diez ejercicios: 10.1 a 10.4 básicos, 10.5 a 10.8 intermedios, 10.9 y 10.10 de desafío. El 10.1 y el 10.8 usan la liga; los demás, la base del caso (`datos/estampa_sur.db`). Las tablas del caso que se usan, con sus claves (el esquema completo está en `datos/esquema.sql`):

Tabla	Columnas	Apunta a
`tienda`	`id_tienda`, `nombre`, `comuna`	
`producto`	`id_producto`, `nombre`, `precio`, `costo_var`, `seg_estampado`	
`proveedor`	`id_proveedor`, `nombre`, `rubro`	
`suministra`	`id_proveedor`, `id_producto`, `costo_unidad`	proveedor y producto
`cliente`	`id_cliente`, `nombre`, `tipo`, `id_tienda`	tienda
`pedido`	`id_pedido`, `id_cliente`, `fecha`, `estado`	cliente
`detalle_pedido`	`id_pedido`, `id_producto`, `cantidad`, `precio_unit`	pedido y producto
`orden_produccion`	`id_orden`, `id_producto`, `semana`, `lote`	producto

10.1 · Contar antes de juntar (básico). (a) ¿Cuántas filas da `SELECT * FROM jugador, gol` en la liga? Dilo antes de correrlo y compruébalo con `COUNT(*)`. (b) ¿Y con la condición de junta correcta? (c) Lo mismo en el caso con `cliente` y `tienda`. ¿Qué número tendría que dar la versión con condición, sin correrla?

10.2 · Quién vende qué (básico). Lista cada producto con cada proveedor que lo suministra y su costo por unidad, ordenado por producto (en el orden de `id_producto`) y, dentro de cada producto, del más barato al más caro.

10.3 · Pedidos por estado (básico). ¿Cuántos pedidos hay en cada estado? Ordena de mayor a menor.

10.4 · Proveedores y cuántos productos ofrecen (básico). Lista cada proveedor con la cantidad de productos que suministra, **incluidos los que no suministran ninguno**. Después cambia `COUNT(s.id_producto)` por `COUNT(*)` y explica la diferencia.

10.5 · Horas de estampado por semana (intermedio). Con `orden_produccion` y `producto`, calcula las horas de estampado que exige cada una de las 8 semanas del plan maestro (lote por segundos de estampado, dividido por 3.600). ¿Qué semana es la más cargada? Compáralo con la capacidad de 40 horas semanales de estampado del caso de Optimización.

10.6 · Tiendas con movimiento (intermedio). ¿Qué tiendas recibieron más de 20 pedidos en marzo? Muestra la tienda y la cantidad.

10.7 · Lo que ya se entregó (intermedio). Calcula las ventas de cada producto considerando solo los pedidos **entregados**, y muestra solo los productos cuyas ventas entregadas superan los $3.000.000. ¿Qué condición va en el `WHERE` y cuál en el `HAVING`, y por qué?

10.8 · Tres consultas de la liga que mienten (intermedio). Para cada una, di qué devuelve, por qué está mal y corrígela.

(a) «Goles por equipo, con el nombre del jugador que los hizo»:

```
SELECT e.nombre AS equipo, j.nombre AS jugador, COUNT(*) AS goles
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_equipo
GROUP BY e.nombre
ORDER BY equipo;
```

(b) «Goles por equipo»:

```
SELECT e.nombre AS equipo, COUNT(*) AS goles
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_jugador
GROUP BY e.nombre
ORDER BY goles DESC;
```

(c) «Jugadores inscritos por equipo»:

```
SELECT e.nombre AS equipo, COUNT(*) AS jugadores
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
GROUP BY e.nombre
ORDER BY e.nombre;
```

10.9 · ¿Cuántos pedidos hizo cada tipo de cliente? (desafío). Un analista escribió:

```
SELECT c.tipo, COUNT(*) AS pedidos, SUM(d.cantidad) AS unidades
FROM pedido p
JOIN cliente c        ON c.id_cliente = p.id_cliente
JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
GROUP BY c.tipo
ORDER BY c.tipo;
```

(a) Córrela y suma la columna `pedidos`. ¿Cuánto debería dar? (b) Explica qué cuenta en realidad. (c) Corrígela y agrega las unidades por pedido de cada tipo, con un decimal. (d) ¿La columna `unidades` también estaba mal?

10.10 · El reporte por tienda (desafío). La gerencia quiere, para **cada una de las 6 tiendas**: clientes inscritos, clientes que compraron, pedidos y ventas de marzo, ordenado por ventas. (a) Escribe la consulta con una sola cadena de `LEFT JOIN` desde `tienda`. (b) ¿Por qué cada conteo necesita `DISTINCT`? (c) Comprueba que las ventas cuadran con `u10_ventas_por_tienda` (sección 14) y que los compradores suman 26. (d) ¿Qué tienda vende más por pedido?

---

13. Pautas

10.1. (a) 10 jugadores por 13 goles:

```
SELECT COUNT(*) AS filas FROM jugador, gol;
```

filas
130

(b) Con la condición queda una fila por gol, porque cada gol tiene un solo autor:

```
SELECT COUNT(*) AS filas FROM jugador j, gol g WHERE g.id_jugador = j.id_jugador;
```

filas
13

(c)

```
SELECT COUNT(*) AS filas FROM cliente, tienda;
```

filas
180

```
SELECT COUNT(*) AS filas FROM cliente c, tienda t WHERE t.id_tienda = c.id_tienda;
```

filas
30

Sin correrla se sabe que da 30: se junta desde el lado «muchos» (`cliente`), cada cliente tiene una tienda y `id_tienda` es `NOT NULL`. Ese número esperado es el control que detecta un join mal escrito.

10.2.

```
SELECT pr.nombre AS producto, pv.nombre AS proveedor, s.costo_unidad
FROM suministra s
JOIN producto pr  ON pr.id_producto = s.id_producto
JOIN proveedor pv ON pv.id_proveedor = s.id_proveedor
ORDER BY pr.id_producto, s.costo_unidad;
```

producto	proveedor	costo_unidad
poleras básicas	Telas Norte	2400
poleras básicas	Algodones Sur	2430
polerones	Algodones Sur	5400
poleras estampado doble	Textil Ñuble	3600
bolsas	Telas Norte	1350
gorros	Textil Ñuble	1880
gorros	Algodones Sur	1910

`suministra` es la tabla intermedia de una relación muchos a muchos (unidad 4): por eso se parte de ella y se junta hacia sus dos lados. Las poleras básicas y los gorros tienen dos proveedores; elegir el más barato **para cada producto** es una subconsulta correlacionada (unidad 11).

10.3.

```
SELECT estado, COUNT(*) AS pedidos
FROM pedido
GROUP BY estado
ORDER BY pedidos DESC;
```

estado	pedidos
entregado	81
en taller	31
pendiente	8

Suman 120, los pedidos de la tabla. Los 8 pendientes son los de la unidad 9; el `GROUP BY` da en una consulta lo que antes pedía una por estado.

10.4.

```
SELECT pv.nombre AS proveedor, COUNT(s.id_producto) AS productos
FROM proveedor pv
LEFT JOIN suministra s ON s.id_proveedor = pv.id_proveedor
GROUP BY pv.id_proveedor, pv.nombre
ORDER BY productos DESC, proveedor;
```

proveedor	productos
Algodones Sur	3
Telas Norte	2
Textil Ñuble	2
Tintas del Bío	0

```
SELECT pv.nombre AS proveedor, COUNT(*) AS productos
FROM proveedor pv
LEFT JOIN suministra s ON s.id_proveedor = pv.id_proveedor
GROUP BY pv.id_proveedor, pv.nombre
ORDER BY productos DESC, proveedor;
```

proveedor	productos
Algodones Sur	3
Telas Norte	2
Textil Ñuble	2
Tintas del Bío	1

Con `COUNT(*)`, Tintas del Bío «suministra 1 producto»: cuenta la fila llena de `NULL` que agregó el `LEFT JOIN`. Tintas del Bío vende tinta, que no es un producto terminado de la tabla `producto`: el modelo no tiene dónde registrar insumos, y eso es una pregunta de diseño (unidad 2), no de SQL.

10.5.

```
SELECT o.semana, SUM(o.lote * pr.seg_estampado) / 3600.0 AS horas_estampado
FROM orden_produccion o
JOIN producto pr ON pr.id_producto = o.id_producto
GROUP BY o.semana
ORDER BY o.semana;
```

semana	horas_estampado
1	9.0
2	20.0
3	15.0
4	18.0
5	15.0
6	33.0
7	17.0
8	18.0

La semana 6 es la más cargada, con 33 h, y queda bajo las 40 h de estampado. Es la misma cifra que usa Gestión de Operaciones para la programación de esa semana (la unidad 11 la desglosa por producto). El `3600.0` evita la división entera; con `3600` la semana 6 daría igual 33, pero una semana de 9,5 h daría 9.

10.6.

```
SELECT t.nombre AS tienda, COUNT(*) AS pedidos
FROM pedido p
JOIN cliente c ON c.id_cliente = p.id_cliente
JOIN tienda t  ON t.id_tienda = c.id_tienda
GROUP BY t.nombre
HAVING COUNT(*) > 20
ORDER BY pedidos DESC;
```

tienda	pedidos
Centro	29
Puerto	23
Valle	22

Aquí `COUNT(*)` sí sirve: se junta desde `pedido` (lado «muchos») y no hay detalle, así que cada fila es un pedido. Centro tiene más pedidos, pero la tienda que más vende es Valle (sección 14): más pedidos no es más venta.

10.7.

```
SELECT pr.nombre AS producto, SUM(d.cantidad * d.precio_unit) AS ventas_entregadas
FROM detalle_pedido d
JOIN pedido p    ON p.id_pedido = d.id_pedido
JOIN producto pr ON pr.id_producto = d.id_producto
WHERE p.estado = 'entregado'
GROUP BY pr.nombre
HAVING SUM(d.cantidad * d.precio_unit) > 3000000
ORDER BY ventas_entregadas DESC;
```

producto	ventas_entregadas
polerones	11571300
poleras estampado doble	4704920
gorros	3636600

`p.estado = 'entregado'` es una condición sobre cada pedido: va en el `WHERE`, antes de agrupar. «Ventas sobre $3.000.000» es una condición sobre la suma de un grupo: va en el `HAVING`. Los polerones entregados suman $11.571.300, más que los otros dos juntos ($4.704.920 y $3.636.600).

10.8. (a) Agrupa por equipo pero muestra un jugador:

```
SELECT e.nombre AS equipo, j.nombre AS jugador, COUNT(*) AS goles
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_equipo
GROUP BY e.nombre
ORDER BY equipo;
```

equipo	jugador	goles
El Bajo	Joaquín Reyes	4
Los Robles	Felipe Sáez	4
Villa Alegre	Tomás Vidal	5

Parece decir que Joaquín Reyes hizo 4 goles, y hizo 3: el 4 es del equipo y el nombre es el de alguna fila del grupo. MySQL rechazaría la consulta (error 1055). Si se quiere el detalle por jugador, el jugador va en el `GROUP BY`:

```
SELECT e.nombre AS equipo, j.nombre AS jugador, COUNT(*) AS goles
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_equipo
GROUP BY e.nombre, j.id_jugador, j.nombre
ORDER BY equipo, goles DESC, jugador;
```

equipo	jugador	goles
El Bajo	Joaquín Reyes	3
El Bajo	Martín Cofré	1
Los Robles	Felipe Sáez	3
Los Robles	Ignacio Lagos	1
Villa Alegre	Tomás Vidal	4
Villa Alegre	Diego Parra	1

(b) La condición junta el equipo con el **número del jugador**:

```
SELECT e.nombre AS equipo, COUNT(*) AS goles
FROM gol g
JOIN jugador j ON j.id_jugador = g.id_jugador
JOIN equipo e  ON e.id_equipo = j.id_jugador
GROUP BY e.nombre
ORDER BY goles DESC;
```

equipo	goles
Las Termas	4
Villa Alegre	3
El Bajo	1

El jugador 4 (Tomás Vidal) quedó en el equipo 4 (Las Termas), el 2 (Sáez) en Villa Alegre y así. Corre sin error y da números enteros y creíbles; solo la sabe detectar quien sabe que Las Termas no tiene jugadores. La corrección es `ON e.id_equipo = j.id_equipo`, que da la tabla de la sección 7.

(c) Con `COUNT(*)`, Las Termas tiene «1 jugador»:

```
SELECT e.nombre AS equipo, COUNT(*) AS jugadores
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
GROUP BY e.nombre
ORDER BY e.nombre;
```

equipo	jugadores
El Bajo	3
Las Termas	1
Los Robles	3
Villa Alegre	3

```
SELECT e.nombre AS equipo, COUNT(j.id_jugador) AS jugadores
FROM equipo e
LEFT JOIN jugador j ON j.id_equipo = e.id_equipo
GROUP BY e.nombre
ORDER BY e.nombre;
```

equipo	jugadores
El Bajo	3
Las Termas	0
Los Robles	3
Villa Alegre	3

Lectura común: ninguna de las tres da error en SQLite. Antes de publicar un número, compáralo con algo que sepas por otro lado (cuántos goles hubo, quién no tiene jugadores).

10.9. (a)

```
SELECT c.tipo, COUNT(*) AS pedidos, SUM(d.cantidad) AS unidades
FROM pedido p
JOIN cliente c        ON c.id_cliente = p.id_cliente
JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
GROUP BY c.tipo
ORDER BY c.tipo;
```

tipo	pedidos	unidades
club	48	2133
colegio	52	2259
empresa	52	2271
persona	60	193

La columna `pedidos` suma 212, y hay 120 pedidos. (b) Al juntar `pedido` con `detalle_pedido` (del «uno» hacia el «muchos») cada pedido se repite una vez por cada producto que lleva, así que `COUNT(*)` cuenta las 212 líneas de detalle, no los pedidos. (c)

```
SELECT c.tipo, COUNT(DISTINCT p.id_pedido) AS pedidos, SUM(d.cantidad) AS unidades,
       ROUND(1.0 * SUM(d.cantidad) / COUNT(DISTINCT p.id_pedido), 1) AS unidades_por_pedido
FROM pedido p
JOIN cliente c        ON c.id_cliente = p.id_cliente
JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
GROUP BY c.tipo
ORDER BY c.tipo;
```

tipo	pedidos	unidades	unidades_por_pedido
club	26	2133	82.0
colegio	26	2259	86.9
empresa	33	2271	68.8
persona	35	193	5.5

Ahora suman 120. (d) No: `cantidad` es un dato de cada línea, y cada línea aparece una sola vez, así que su suma está bien. Lo que se infla al juntar hacia el «muchos» es lo que pertenece al lado «uno»: contar pedidos, o sumar un dato del pedido (un costo de despacho, por ejemplo) se multiplicaría por la cantidad de líneas. **Lectura:** las personas hacen 35 pedidos, más que cualquier otro tipo, pero de 5,5 unidades en promedio, contra 68,8 de las empresas, 82,0 de los clubes y 86,9 de los colegios. Son muchos pedidos chicos: el costo de atenderlos (unidad de Contabilidad: costos por actividad) puede comerse su margen.

10.10. (a)

```
SELECT t.nombre AS tienda,
       COUNT(DISTINCT c.id_cliente) AS clientes,
       COUNT(DISTINCT p.id_cliente) AS compradores,
       COUNT(DISTINCT p.id_pedido) AS pedidos,
       COALESCE(SUM(d.cantidad * d.precio_unit), 0) AS ventas
FROM tienda t
LEFT JOIN cliente c        ON c.id_tienda = t.id_tienda
LEFT JOIN pedido p         ON p.id_cliente = c.id_cliente
LEFT JOIN detalle_pedido d ON d.id_pedido = p.id_pedido
GROUP BY t.id_tienda, t.nombre
ORDER BY ventas DESC;
```

tienda	clientes	compradores	pedidos	ventas
Valle	5	4	22	7297650
Centro	5	5	29	6951140
Lagos	5	4	16	6134350
Costa	5	4	17	5922540
Puerto	5	5	23	5534370
Cerro	5	4	13	3883390

(b) La cadena va siempre del «uno» al «muchos»: cada cliente se repite por cada uno de sus pedidos, y cada pedido por cada línea de detalle. Sin `DISTINCT`, «clientes» contaría líneas. `p.id_cliente` (y no `c.id_cliente`) cuenta compradores, porque vale `NULL` en el cliente sin pedidos. El `SUM` no necesita `DISTINCT`: la cantidad es de la línea. `COALESCE` deja 0 si una tienda no tuviera ventas. (c) Las ventas son las de la sección 14, tienda por tienda, y suman $35.723.440; los compradores suman 26 (los 30 clientes menos los 4 sin pedidos). (d) Valle: $7.297.650 en 22 pedidos, unos $331.711 por pedido, contra $239.694 de Centro. Centro tiene más pedidos y vende menos. Lagos, Valle, Costa y Cerro tienen cada una un cliente inscrito que no ha comprado: son los 4 de la sección 14.

14. Caso integrador · Estampa Sur junta sus tablas

> La unidad 9 le hizo a la base de Estampa Sur preguntas de una tabla. Ahora las preguntas cruzan tablas: las ventas viven en `detalle_pedido`, pero la tienda está tres claves foráneas más allá. Las consultas están en `sql/u10_caso.sql` y son las mismas del script del caso.

El costo de olvidar la condición.

```
SELECT COUNT(*) AS filas FROM cliente, pedido;
```

filas
3600

3.600 filas para 120 pedidos. Cada pedido aparece 30 veces, una con cada cliente.

Ventas por tienda. El camino es `detalle_pedido` → `pedido` → `cliente` → `tienda`: cuatro tablas, tres condiciones.

```
SELECT t.nombre, SUM(d.cantidad * d.precio_unit) AS ventas
FROM detalle_pedido d
JOIN pedido p   ON p.id_pedido = d.id_pedido
JOIN cliente c  ON c.id_cliente = p.id_cliente
JOIN tienda t   ON t.id_tienda = c.id_tienda
GROUP BY t.nombre ORDER BY ventas DESC;
```

nombre	ventas
Valle	7297650
Centro	6951140
Lagos	6134350
Costa	5922540
Puerto	5534370
Cerro	3883390

Unidades por producto.

```
SELECT pr.nombre, SUM(d.cantidad) AS unidades
FROM detalle_pedido d JOIN producto pr ON pr.id_producto = d.id_producto
GROUP BY pr.nombre ORDER BY unidades DESC;
```

nombre	unidades
gorros	1746
polerones	1449
poleras estampado doble	1379
bolsas	1304
poleras básicas	978

Margen por tipo de cliente, solo los tipos que compraron más de 100 unidades. El margen de cada línea es la cantidad por (precio cobrado menos costo variable); el costo está en `producto`, así que hace falta una cuarta tabla.

```
SELECT c.tipo, SUM(d.cantidad * (d.precio_unit - pr.costo_var)) AS margen
FROM detalle_pedido d
JOIN pedido p    ON p.id_pedido = d.id_pedido
JOIN cliente c   ON c.id_cliente = p.id_cliente
JOIN producto pr ON pr.id_producto = d.id_producto
GROUP BY c.tipo HAVING SUM(d.cantidad) > 100 ORDER BY margen DESC;
```

tipo	margen
empresa	4667900
colegio	4427520
club	4004330
persona	383940

El `HAVING` no eliminó a nadie. Para saber por qué, se miran las unidades que filtra:

```
SELECT c.tipo, SUM(d.cantidad) AS unidades
FROM detalle_pedido d
JOIN pedido p  ON p.id_pedido = d.id_pedido
JOIN cliente c ON c.id_cliente = p.id_cliente
GROUP BY c.tipo
ORDER BY unidades DESC;
```

tipo	unidades
empresa	2271
colegio	2259
club	2133
persona	193

Las personas compraron 193 unidades: pasan el umbral de 100. Un `HAVING` que no filtra nada no es un error, pero conviene comprobarlo; con un umbral de 1.000, las personas quedarían fuera.

Clientes sin pedidos.

```
SELECT COUNT(*) FROM cliente c LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.id_pedido IS NULL;
```

COUNT(*)
4

¿Quiénes son?

```
SELECT c.nombre, c.tipo, t.nombre AS tienda
FROM cliente c
JOIN tienda t ON t.id_tienda = c.id_tienda
LEFT JOIN pedido p ON p.id_cliente = c.id_cliente
WHERE p.id_pedido IS NULL
ORDER BY c.id_cliente;
```

nombre	tipo	tienda
Cliente 27	persona	Lagos
Cliente 28	persona	Valle
Cliente 29	persona	Costa
Cliente 30	persona	Cerro

Lectura.

- **Las ventas de marzo suman $35.723.440.** Valle encabeza con $7.297.650 (20,4 % del total) y Cerro cierra con $3.883.390, poco más de la mitad. Las seis tiendas tienen 5 clientes cada una: la diferencia está en cuánto compra cada cliente, no en cuántos hay.
- **Los gorros son lo que más unidades mueve (1.746), pero no lo que más deja.** Su margen unitario es $1.170 (unidad 9), menos de un tercio del de un polerón.
- **El margen viene de los clientes institucionales.** Empresas ($4.667.900), colegios ($4.427.520) y clubes ($4.004.330) dejan casi todo; las personas, $383.940. Si hay que priorizar la atención comercial, va a las empresas y colegios.
- **Los 4 clientes sin pedidos son personas**, uno por tienda en Lagos, Valle, Costa y Cerro: se inscribieron y no volvieron. Es una lista concreta para llamar.

La cadena. El verificador corre `caso_estampa_sur_bd.py`, ejecuta `u10_caso.sql` y exige que las cuatro consultas `u10_*` sean las del script del caso y den lo mismo (`CADENA: OK`). Además rehace en Python cada junta y cada grupo desde las filas crudas, y cruza los totales: las ventas por tienda suman lo mismo que el reporte del ejercicio 10.10, y los pedidos por tipo suman 120.

Lo que pasa a la unidad siguiente. Hay preguntas que un join con `GROUP BY` no responde de una vez: «¿qué clientes compraron más que el promedio?» necesita primero el promedio, y «¿cuál es el proveedor más barato de cada producto?» necesita comparar cada fila con el mínimo de su grupo. La unidad 11 las resuelve con subconsultas, y guarda las consultas de esta unidad como vistas.

15. Para seguir

En los libros del área (el programa del ramo no fija bibliografía):

- Elmasri y Navathe, *Fundamentos de sistemas de bases de datos*: el capítulo de SQL avanzado trata las juntas internas y externas, las funciones de agregación, `GROUP BY` y `HAVING`.
- Silberschatz, Korth y Sudarshan, *Fundamentos de bases de datos*: los capítulos de SQL distinguen la junta natural, las juntas externas y la agregación con `NULL`.

Abiertos y gratis.

- SQLite, «SELECT» (en inglés): la sintaxis de las juntas en SQLite y la sección sobre columnas sueltas en una consulta agregada («Bare columns in an aggregate query»). https://www.sqlite.org/lang_select.html
- MySQL 8.4 Reference Manual, sección 14.19.3 «MySQL Handling of GROUP BY» (en inglés): por qué MySQL rechaza las columnas que no están en el `GROUP BY` y cuándo las acepta por dependencia funcional. https://dev.mysql.com/doc/refman/8.4/en/group-by-handling.html

Con el tutor. La carpeta `tutor/` trae el contexto de este ramo. Pégalo al inicio de la conversación con cualquier IA y pídele, por ejemplo: «Escríbeme tres consultas con `JOIN` y `GROUP BY` sobre la base de Estampa Sur, una correcta y dos con un error que no dé mensaje; yo tengo que decir cuál es cuál y por qué».
